

作业 HW3* 实验报告

姓名：汪嘉晨 学号：2353900 日期：2024 年 11 月 11 日

1. 涉及数据结构和相关背景

一、树的数据结构

1. 树的基本概念：

- 树是由 n ($n \geq 0$) 个节点（或称为顶点）组成的有限集合。
- 如果 $n=0$ ，则称为空树。
- 如果 $n>0$ ，则存在一个特定的节点，称为根节点，其余节点可分为 m ($m \geq 0$) 个互不相交的有限集，其中每个集合又是一棵树，并称为根的子树。

2. 树的相关术语：

- **根节点**：树中唯一的没有父节点的节点。
- **叶子节点（叶节点）**：没有子节点的节点。
- **内部节点（非叶子节点）**：除根节点和叶子节点之外的节点。
- **边**：连接两个节点的线段。
- **子树**：以某个节点为根的子节点及其所有子节点组成的树。
- **树的深度（高度）**：树中节点的最大层次数（根节点的层次通常定义为 1 或 0，这里存在不同定义，但不影响对树深度的理解）。
- **节点的度**：一个节点含有的子树的个数。
- **树的度**：树中节点的最大度数。

3. 树的基本操作：

- **树的创建**：在空树中添加节点，形成一棵具有层次关系的树。
- **插入节点**：在树中添加新的节点，插入节点的位置可以根据某种规则确定。
- **删除节点**：在树中删除指定的节点，并调整树的结构以保持树的形状。
- **查找节点**：在树中根据关键字或其它特定条件寻找指定的节点。
- **遍历树**：按照某种规则访问树中的每个节点，并且每个节点仅被访问一次。常见的遍历方式有先序遍历、中序遍历、后序遍历和层次遍历。

二、二叉树的数据结构

1. 二叉树的基本概念：

- 二叉树是树形结构的一个重要类型，每个节点最多只能有两棵子树，分别称为左子树和右子树，且左右子树是有顺序的，次序不能任意颠倒。
- 二叉树是 n ($n \geq 0$) 个有限元素的集合，该集合或者为空（空二叉树），或者由一个称为根（root）的元素及两个不相交的、被分别称为左子树和右子树的二叉树组成。

2. 特殊的二叉树：

- **满二叉树**：如果一棵二叉树只有度为 0 的节点和度为 2 的节点，并且度为 0 的节点在同一层上，则这棵二叉树为满二叉树。在满二叉树中，每一层的节点数都达到最大值。
- **完全二叉树**：深度为 k ，有 n 个节点的二叉树，当且仅当其每一个节点都与深度为 k 的满二叉树中编号从 1 到 n 的节点一一对应时，称为完全二叉树。在完全二叉树中，叶子节点只能出现在最下层和次下层，且最下面一层的节点

都集中在该层最左边的若干位置。

3. 二叉树的存储结构：

- **顺序存储**：使用一维数组来存储二叉树节点中的数据，节点的编号位置就是数组的下标索引。但这种方法只适合满二叉树或完全二叉树，对于不完全二叉树会造成空间浪费。

- **链式存储**：使用链表来表示二叉树，节点之间通过指针来连接。每个节点通常包含数据域、指向左子树的指针和指向右子树的指针。

4. 二叉树的基本操作：

- **二叉树的创建**：使用递归或迭代的方式根据输入数据构建二叉树。

- **遍历二叉树**：按照某种规则访问二叉树中的每个节点。常见的遍历方式有先序遍历、中序遍历、后序遍历和层次遍历。

- **查找节点**：在二叉树中查找具有特定值的节点。对于二叉搜索树 (BST)，查找过程类似于二分查找。

- **插入节点**：在二叉树中插入新的节点，并保持二叉树的性质（如二叉搜索树的性质）。

- **删除节点**：从二叉树中删除指定的节点，并重新调整树的结构以保持二叉树的性质。

2. 实验内容

2.1 二叉树的非递归遍历

2.1.1 问题描述

二叉树的非递归遍历可通过栈来实现。例如对于由 `abc##d##ef###` 先序建立的二叉树，如下图 1 所示，中序非递归遍历（参照课本 p131 算法 6.3）可以通过如下系列栈的入栈出栈操作来完成：`push(a) push(b) push(c) pop pop push(d) pop pop push(e) push(f) pop pop`。

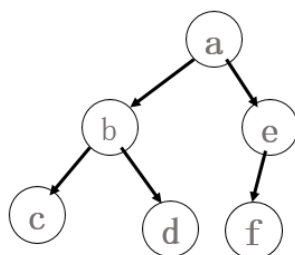


图 1

如果已知中序遍历的栈的操作序列，就可唯一地确定一棵二叉树。请编程输出该二叉树的后序遍历序列。

提示：本题有多种解法，仔细分析二叉树非递归遍历过程中栈的操作规律与遍历序列的关系，可将二叉树构造出来。

2.1.2 基本要求

输入

第一行一个整数 n ，表示二叉树的结点个数。

接下来 $2n$ 行，每行描述一个栈操作，格式为：`push X` 表示将结点 X 压入栈中，`pop` 表示从栈

中弹出一个结点。
(X 用一个字符表示)
对于 20%的数据, $0 < n \leq 10$
对于 40%的数据, $0 < n \leq 20$
对于 100%的数据, $0 < n \leq 83$
从右上角下载并运行 p37_data.cpp 以生成随机测试数据
输出
一行, 后序遍历序列。

2.1.3 数据结构设计

```
// 定义二叉树节点的结构体
struct TreeNode
{
    char value; // 节点的值
    TreeNode* left; // 左子节点指针
    TreeNode* right; // 右子节点指针
    // 构造函数, 使用初始化列表初始化成员变量
    TreeNode(char val) : value(val), left(nullptr), right(nullptr) {}
};
```

2.1.4 功能说明 (函数、类)

```
// 后序遍历函数, 递归实现
void postOrderTraversal(TreeNode* node, vector<char>& result)
{
    if (node == nullptr)
        return; // 如果节点为空, 直接返回
    postOrderTraversal(node->left, result); // 递归遍历左子树
    postOrderTraversal(node->right, result); // 递归遍历右子树
    result.push_back(node->value); // 访问当前节点
    // 读取 2n 行的栈操作
    for (int i = 0; i < 2 * n; ++i)
    {
        string operation;
        cin >> operation; // 读取操作类型
        if (operation == "push")
        {
            char value;
            cin >> value; // 读取节点值
            TreeNode* node = new TreeNode(value); // 创建新节点
            if (root == nullptr)
            {
                root = node; // 如果根节点为空, 则将新节点设为根节点
            }
        }
    }
}
```

```

    }
    else
    {
        if (current->left == nullptr)
        {
            current->left = node; // 如果当前节点的左子节点为空，则将新节点设为
左子节点
        }
        else
        {
            current->right = node; // 否则将新节点设为右子节点
        }
    }
    s.push(node); // 将新节点压入栈中
    current = node; // 更新当前节点
}
else if (operation == "pop")
{
    current = s.top(); // 弹出栈顶节点
    s.pop();
}
}

```

2.1.5 调试分析（遇到的问题和解决方法）

问题：在 `pop` 操作中，当前节点 `current` 被更新为栈顶节点，这可能会导致逻辑错误。

解决方法：在 `pop` 操作中，不需要更新 `current`，只需弹出栈顶节点即可。

2.1.6 总结和体会

内存管理：在使用动态内存分配时，必须确保在适当的时候释放内存，以避免内存泄漏。

输入验证：在处理用户输入时，必须进行验证，以确保输入格式正确，避免未定义行为。

这道题确实好巧妙！

2.2 二叉树的同构

2.2.1 问题描述

给定两棵树 T1 和 T2。如果 T1 可以通过若干次左右孩子互换变成 T2，则我们称两棵树是“同构”的。例如图 1 给出的两棵树就是同构的，因为我们把其中一棵树的结点 a、b、e 的左右孩子互换后，就得到另外一棵树。而图 2 就不是同构的。

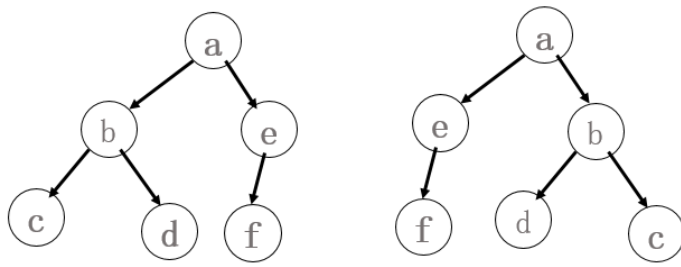


图 1

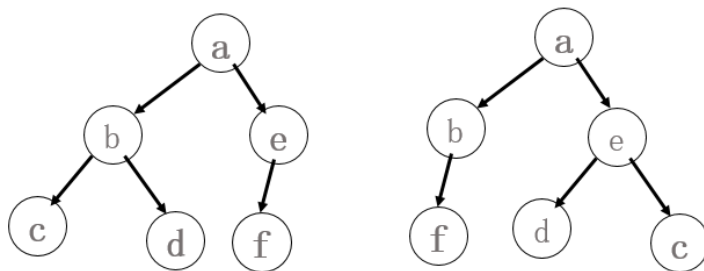


图 2

现给定两棵树，请你判断它们是否是同构的。并计算每棵树的深度。

2.2.2 基本要求

输入：

第一行是一个非负整数 N_1 ，表示第 1 棵树的结点数；

随后 N 行，依次对应二叉树的 N 个结点（假设结点从 0 到 $N-1$ 编号），每行有三项，分别是 1 个英文大写字母、其左孩子结点的编号、右孩子结点的编号。如果孩子结点为空，则在相应位置上给出“-”。给出的数据间用一个空格分隔。

接着一行是一个非负整数 N_2 ，表示第 2 棵树的结点数；

随后 N 行同上描述一样，依次对应二叉树的 N 个结点。

对于 20% 的数据，有 $0 < N_1 = N_2 \leq 10$

对于 40% 的数据，有 $0 \leq N_1 = N_2 \leq 100$

对于 100% 的数据，有 $0 \leq N_1, N_2 \leq 10100$

注意：题目不保证每个结点中存储的字母是不同的。

下载 p81_data.cpp 并编译运行以生成随机测试数据

输出：

共三行。

第一行，如果两棵树是同构的，输出“Yes”，否则输出“No”。

后面两行分别是两棵树的深度。

2.2.3 数据结构设计

// 定义树节点结构

```
struct TreeNode {
    char data; // 节点数据
```

```

        int left, right; // 左右孩子的索引
};

// 定义二叉树节点结构
struct Node {
    char data; // 节点数据
    Node* left; // 左孩子指针
    Node* right; // 右孩子指针
    Node(char d) : data(d), left(nullptr), right(nullptr) {} // 构造函数
};

```

2.2.4 功能说明（函数、类）

```

// 根据节点数组和索引构建二叉树
Node* buildTree(const vector<TreeNode>& nodes, int index) {
    if (index == -1) return nullptr; // 如果索引为-1, 返回空指针
    Node* root = new Node(nodes[index].data); // 创建新节点
    root->left = buildTree(nodes, nodes[index].left); // 递归构建左子树
    root->right = buildTree(nodes, nodes[index].right); // 递归构建右子树
    return root; // 返回根节点
}

// 判断两棵树是否同构
bool isIsomorphic(Node* T1, Node* T2)
{
    if (T1 == nullptr && T2 == nullptr) return true; // 如果两棵树都为空, 返回 true
    if (T1 == nullptr || T2 == nullptr) return false; // 如果其中一棵树为空, 返回 false
    if (T1->data != T2->data) return false; // 如果根节点数据不同, 返回 false
    // 递归判断左右子树是否同构
    return (isIsomorphic(T1->left, T2->left) && isIsomorphic(T1->right, T2->right)) ||
           (isIsomorphic(T1->left, T2->right) && isIsomorphic(T1->right, T2->left));
}

// 计算二叉树的深度
int getDepth(Node* root) {
    if (root == nullptr) return 0; // 如果节点为空, 深度为 0
    // 递归计算左右子树的深度, 返回较大值加 1
    return max(getDepth(root->left), getDepth(root->right)) + 1;
}

// 找到树的根节点
int findRoot(const vector<TreeNode>& nodes) {
    int n = nodes.size();
    vector<bool> isChild(n, false); // 标记节点是否为孩子节点
    for (const auto& node : nodes) {

```

```

        if (node.left != -1) isChild[node.left] = true; // 标记左孩子
        if (node.right != -1) isChild[node.right] = true; // 标记右孩子
    }
    for (int i = 0; i < n; ++i) {
        if (!isChild[i]) return i; // 找到没有被标记为孩子的节点，即为根节点
    }
    return -1; // 如果没有找到根节点，返回-1
}

int main() {
    int N1, N2;
    cin >> N1; // 输入第一棵树的节点数
    vector<TreeNode> nodes1(N1);
    for (int i = 0; i < N1; ++i) {
        char data;
        string left, right;
        cin >> data >> left >> right; // 输入节点数据和左右孩子
        nodes1[i] = { data, left == "-" ? -1 : stoi(left), right == "-" ? -1 : stoi(right) };
    }
    cin >> N2; // 输入第二棵树的节点数
    vector<TreeNode> nodes2(N2);
    for (int i = 0; i < N2; ++i) {
        char data;
        string left, right;
        cin >> data >> left >> right; // 输入节点数据和左右孩子
        nodes2[i] = { data, left == "-" ? -1 : stoi(left), right == "-" ? -1 : stoi(right) };
    }

    int root1 = findRoot(nodes1); // 找到第一棵树的根节点
    int root2 = findRoot(nodes2); // 找到第二棵树的根节点
    Node* T1 = buildTree(nodes1, root1); // 构建第一棵树
    Node* T2 = buildTree(nodes2, root2); // 构建第二棵树

    if (isIsomorphic(T1, T2)) {
        cout << "Yes" << endl;
    }
    else {
        cout << "No" << endl;
    }
    cout << getDepth(T1) << endl << getDepth(T2) << endl;
    return 0;
}

```

2.2.5 调试分析（遇到的问题 and 解决方法）

问题：如果输入的栈操作顺序不正确，可能会导致构造的二叉树不符合预期。

解决方法：确保输入的栈操作顺序正确，并在代码中添加适当的检查和错误处理。

2.2.6 总结和体会

1. 二叉树的构建

- 数据结构：使用 `TreeNode` 结构体存储节点信息，包括节点数据和左右孩子的索引。使用 `Node` 结构体构建二叉树，包括节点数据和左右孩子的指针。

- 递归构建：通过递归函数 `buildTree` 根据节点数组和索引构建二叉树。递归的终止条件是索引为 `-1`，表示当前节点为空。

2. 同构性判断

- 递归判断：通过递归函数 `isIsomorphic` 判断两棵树是否同构。两棵树同构的条件是：
 - 两棵树都为空。
 - 其中一棵树为空，另一棵树不为空。
 - 根节点数据相同，且左右子树同构（包括左右子树交换的情况）。

3. 树的深度计算

- 递归计算：通过递归函数 `getDepth` 计算二叉树的深度。递归的终止条件是节点为空，深度为 `0`。递归计算左右子树的深度，返回较大值加 `1`。

4. 根节点的查找

- 标记法：通过 `findRoot` 函数找到树的根节点。遍历节点数组，标记每个节点的左右孩子。没有被标记为孩子的节点即为根节点。

5. 输入输出处理

- 输入格式：根据题目要求，输入两棵树的节点数和节点信息。节点信息包括节点数据和左右孩子的索引（用 `-1` 表示空节点）。
- 输出结果：输出两棵树是否同构的结果，以及两棵树的深度。

体会

- 递归思想：递归是解决树结构问题的常用方法。通过递归函数，可以简洁地实现树的构建、遍历、深度计算等操作。
- 边界条件：处理树结构问题时，需要特别注意边界条件，如空节点的处理。
- 数据结构设计：合理的数据结构设计可以简化问题的解决过程。本题中，使用两个结构体分别存储输入数据和构建二叉树，便于操作和理解。

2.3 感染二叉树需要的总时间

2.3.1 问题描述

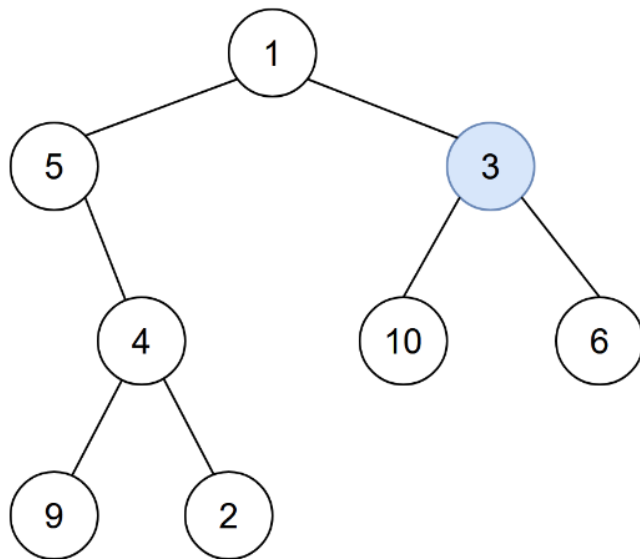
感染二叉树需要的总时间

给你一棵二叉树的根节点 `root`，二叉树中节点的值 **互不相同**。另给你一个整数 `start`。在第 `0` 分钟，**感染** 将会从值为 `start` 的节点开始爆发。

每分钟，如果节点满足以下全部条件，就会被感染：

- 节点此前还没有感染。
- 节点与一个已感染节点相邻。

返回感染整棵树需要的分钟数。



例如上图中 $start = 3$ ，输出：4

解释：节点按以下过程被感染： - 第 0 分钟：节点 3 - 第 1 分钟：节点 1、10、6 - 第 2 分钟：节点 5 - 第 3 分钟：节点 4 - 第 4 分钟：节点 9 和 2 感染整棵树需要 4 分钟，所以返回 4。

2.3.2 基本要求

输入

第一行包含两个整数 n 和 $start$

接下来包含 n 行，描述 n 个节点的左、右孩子编号

0 号节点为根节点， $0 \leq start < n$

对于 20% 的数据， $1 \leq n \leq 10$

对于 40% 的数据， $1 \leq n \leq 1000$

对于 100% 的数据， $1 \leq n \leq 100000$

下载编译并运行 p128_data.cpp 生成随机测试数据

输出

一个整数，表示感染整棵二叉树所需要的时间

2.3.3 数据结构设计

// 构建二叉树的邻接表

```
vector<vector<int>> tree(n);
```

```
for (int i = 0; i < n; ++i) {
```

```
    int left, right;
```

```
    cin >> left >> right;
```

```
    if (left != -1) {
```

```
        tree[i].push_back(left);
```

```
        tree[left].push_back(i);
```

```
    }
```

```
    if (right != -1) {
```

```
        tree[i].push_back(right);
```

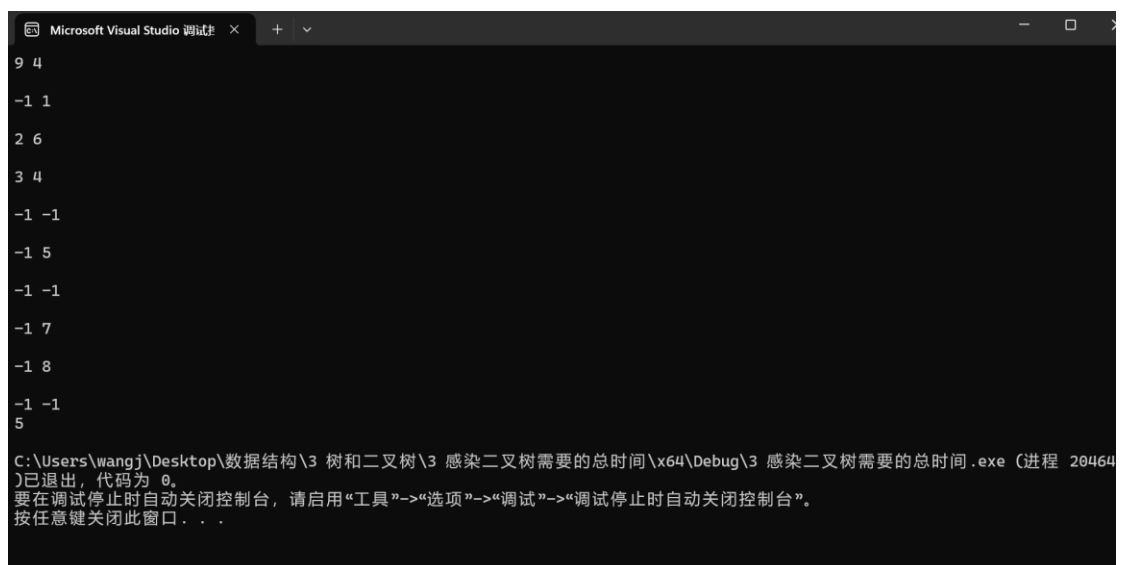
```
        tree[right].push_back(i);
```

```
    }  
}
```

2.3.4 功能说明（函数、类）

```
// BFS 算法  
queue<int> q;  
vector<bool> visited(n, false);  
vector<int> time(n, 0);  
  
q.push(start);  
visited[start] = true;  
time[start] = 0;  
  
int maxTime = 0;  
  
while (!q.empty()) {  
    int node = q.front();  
    q.pop();  
    for (int neighbor : tree[node]) {  
        if (!visited[neighbor]) {  
            visited[neighbor] = true;  
            time[neighbor] = time[node] + 1;  
            maxTime = max(maxTime, time[neighbor]);  
            q.push(neighbor);  
        }  
    }  
}
```

2.3.5 调试分析（遇到的问题解决方法）



2.3.6 总结和体会

通过 BFS 算法有效地解决了从某个节点开始感染整棵二叉树所需时间的问题，具有较高的实用性和可读性。

2.4 树的重构

2.4.1 问题描述

树木在计算机科学中有许多应用。也许最常用的树是二叉树,但也有其他的同样有用的树类型。其中一个例子是有序树（任意节点的子树是有序的）。

每个节点的子节点数是可变的，并且数量没有限制。一般而言，有序树由有限节点集合 T 组成，并且满足：

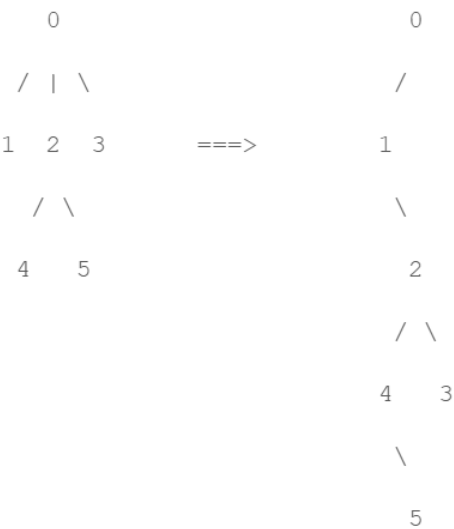
- 1.其中一个节点置为根节点，定义为 $root(T)$;
- 2.其他节点被划分为若干子集 $T_1, T_2, \dots, T_m$, 每个子集都是一个树.

同样定义 $root(T_1), root(T_2), \dots, root(T_m)$ 为 $root(T)$ 的孩子，其中 $root(T_i)$ 是第 i 个孩子。节点 $root(T_1), \dots, root(T_m)$ 是兄弟节点。

通常将一个有序树表示为二叉树是更加有用的，这样每个节点可以存储在相同内存空间中。有序树到二叉树的转化步骤为：

- 1.去除每个节点与其子节点的边
- 2.对于每一个节点，在它与第一个孩子节点（如果存在）之间添加一条边，作为该节点的左孩子
- 3.对于每一个节点，在它与下一个兄弟节点（如果存在）之间添加一条边，作为该节点的右孩子

如图所示：



在大多数情况下，树的深度（从根节点到叶子节点的边数的最大值）都会在转化后增加。这是不希望发生的事情因为很多算法的复杂度都取决于树的深度。

现在，需要你实现一个程序来计算转化前后的树的深度。

2.4.2 基本要求

输入

输入由多行组成，每一行都是一棵树的深度优先遍历时的方向。其中 d 表示下行(down), u 表示上行(up)。

例如上面的树就是 dudduduudu,表示从 0 下行到 1,1 上行到 0,0 下行到 2 等等。输入的截止为以 # 开始的行。

可以假设每棵树至少含有 2 个节点，最多 10000 个节点。

输出

对每棵树，打印转化前后的树的深度，采用以下格式 Tree t: h1 => h2。其中 t 表示样例编号(从 1 开始)，h1 是转化前的树的深度，h2 是转化后的树的深度。

2.4.3 数据结构设计——似乎并没有创建类或者结构体的需要

2.4.4 功能说明（函数、类）

```
int main() {
    int cnt = 0; // 记录树的数量
    char t[1000000]; // 存储输入字符串
    while (cin >> t) {
        if (t[0] == '#')
            break; // 遇到 '#' 结束输入
        int nh = 0, h = 0, nh2 = 0, h2 = 0; // 初始化高度变量
        stack<int> now; // 栈用于记录二叉树的节点高度
        now.push(0); // 初始节点高度为 0
        for (int i = 0; t[i] != '\0'; i++) {
            if (t[i] == 'd') {
                nh++; // 原始树当前高度增加
                nh2++; // 二叉树当前高度增加
                now.push(nh2); // 将当前节点高度压入栈中
            }
            else {
                nh--; // 原始树当前高度减少
                nh2 = now.top(); // 获取栈顶元素，表示当前节点的父节点高度
                now.pop(); // 当前节点出栈
            }
            h2 = max(h2, nh2); // 更新二叉树的最大高度
            h = max(h, nh); // 更新原始树的最大高度
        }
        cout << "Tree " << ++cnt << ": " << h << " => " << h2 << endl; // 输出结果
    }
    return 0;
}
```

2.4.5 调试分析（遇到的问题 and 解决方法）

```
Microsoft Visual Studio 调试 × + -
duddduuudu
Tree 1: 2 => 4

ddddduduuu
Tree 2: 5 => 5

ddddduduuu
Tree 3: 4 => 5

ddddduduuu
Tree 4: 4 => 4

#

C:\Users\wangj\Desktop\数据结构\3 树和二叉树\5 树的重构\x64\Debug\5 树的重构.exe (进程 20732)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . .
```

2.4.6 总结和体会

- **深度优先遍历（DFS）：**
 - 这道题使用了深度优先遍历（DFS）来遍历树。通过字符 d 和 u 分别表示向下和向上遍历，模拟了树的遍历过程。
- **栈的应用：**
 - 使用栈来记录当前节点的高度。每次遇到 d 时，将当前节点的高度压入栈中；每次遇到 u 时，从栈中弹出当前节点的高度。这种方法有效地模拟了树的遍历过程，并且能够方便地获取当前节点的父节点高度。
- **高度计算：**
 - 通过两个变量 nh 和 nh2 分别记录原始树和重构后的二叉树的当前高度，并在遍历过程中不断更新最大高度 h 和 h2。这种方法能够有效地计算树的深度。

2.5 最近公共祖先

2.5.1 问题描述

给出一颗多叉树，请你求出两个节点的最近公共祖先。

一个节点的祖先节点可以是该节点本身，树中任意两个节点都至少有一个共同祖先，即根节点。

2.5.2 基本要求

输入

输入数据包含 T 个测试样本，每个样本 i 包含 N_i 个节点和 N_i-1 条边和 M_i 个问题，树中节点从 1 到 N_i 编号

输入第一行是测试样本数 T

每个测试样本 i 第一行为两个整数 N_i 和 M_i

接下来 N_i-1 行，每行 2 个整数 a 、 b ，表示 a 是 b 的父节点

接下来 M_i 行，每行两个整数 x 、 y ，表示询问 x 和 y 的共同祖先

对于 100%的数据，

$1 \leq T \leq 100$;

$5 \leq N \leq 1000$;

$5 \leq M \leq 1000$;

输出

对于每一个询问输出一个整数，表示共同祖先的编号

2.5.3 数据结构设计

```
class Tree {
public:
    unordered_map<int, int> parent;
    unordered_map<int, unordered_set<int>> children;

    void addEdge(int parent, int child) {
        this->parent[child] = parent;
        this->children[parent].insert(child);
    }

    int findLCA(int x, int y) {
        unordered_set<int> ancestors;
        while (x != 0) {
            ancestors.insert(x);
            x = parent[x];
        }
        while (y != 0) {
            if (ancestors.find(y) != ancestors.end()) {
                return y;
            }
            y = parent[y];
        }
        return -1;
    }
};
```

2.5.4 功能说明（函数、类）

```
void addEdge(int parent, int child) {
    this->parent[child] = parent;
    this->children[parent].insert(child);
}

int findLCA(int x, int y) {
    unordered_set<int> ancestors;
    while (x != 0) {
        ancestors.insert(x);
        x = parent[x];
    }
    while (y != 0) {
        if (ancestors.find(y) != ancestors.end()) {
            return y;
        }
    }
    return -1;
}
```

```

        return y;
    }
    y = parent[y];
}
return -1;
}

```

2.5.5 调试分析（遇到的问题 and 解决方法）

```

Microsoft Visual Studio 调试
2
16 1
1 14
8 5
10 16
5 9
4 6
8 4
4 10
1 13
6 15
10 11
6 7
10 2
16 3
8 1
16 12
16 7
4
5 1
2 3
3 4
3 1
1 5
3 5
3
C:\Users\wangj\Desktop\数据结构\3 树和二叉树\4 最近公共祖先\Debug\4 最近公共祖先.exe (进程 7076)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...

```

2.5.6 总结和体会

首先读取测试样本数 I ，然后对于每个测试样本，读取节点数 N 和查询数 M 。接着读取 $N-1$ 条边来构建树，并读取 M 个查询来查找最近公共祖先。`Tree` 类中包含了一个 `parent` 映射和一个 `children` 映射，用于存储树的结构。`findLCA` 方法用于查找两个节点的最近公共祖先。

2.6 求树的后序

2.6.1 问题描述

给出二叉树的前序遍历和中序遍历，求树的后序遍历

2.6.2 基本要求

输入

输入包含若干行，每一行有两个字符串，中间用空格隔开

同行的两个字符串从左到右分别表示树的前序遍历和中序遍历，由单个字符组成，每个字符表示一个节点

字符仅包括大小写英文字母和数字，最多 62 个

输入保证一颗二叉树内不存在相同的节点

输出

每一行输入对应一行输出

若给出的前序遍历和中序遍历对应存在一棵二叉树，则输出其后序遍历

否则输出 Error

样例输入

DBACEGF ABCDEFG

BCAD CBAD

ABC CAB

样例输出

ACBFGED

CDAB

Error

2.6.3 数据结构设计

递归!!!

2.6.4 功能说明（函数、类）

// 递归函数，用于根据前序和中序遍历构建后序遍历

```
bool BuildPostorder(const string& preorder, const string& inorder, string& postorder) {  
    if (preorder.empty() || inorder.empty()) {  
        return true;  
    }  
}
```

// 前序遍历的第一个节点是根节点

```
char rootValue = preorder[0];
```

```
size_t rootIndex = inorder.find(rootValue);
```

// 如果在中序遍历中找不到根节点，说明输入不合法

```
if (rootIndex == string::npos) {  
    return false;  
}
```

// 递归处理左子树

```
if (!BuildPostorder(preorder.substr(1, rootIndex), inorder.substr(0, rootIndex),  
postorder)) {  
    return false;  
}
```

// 递归处理右子树

```
if (!BuildPostorder(preorder.substr(rootIndex + 1), inorder.substr(rootIndex + 1),  
postorder)) {  
    return false;  
}
```

// 将根节点添加到后序遍历结果中

```
postorder += rootValue;
```

```
return true;
```

```
}
```



```

int main() {
    string input;
    while (getline(cin, input)) {
        size_t spaceIndex = input.find(' ');
        if (spaceIndex == string::npos) {
            cout << "Error" << endl;
            continue;
        }

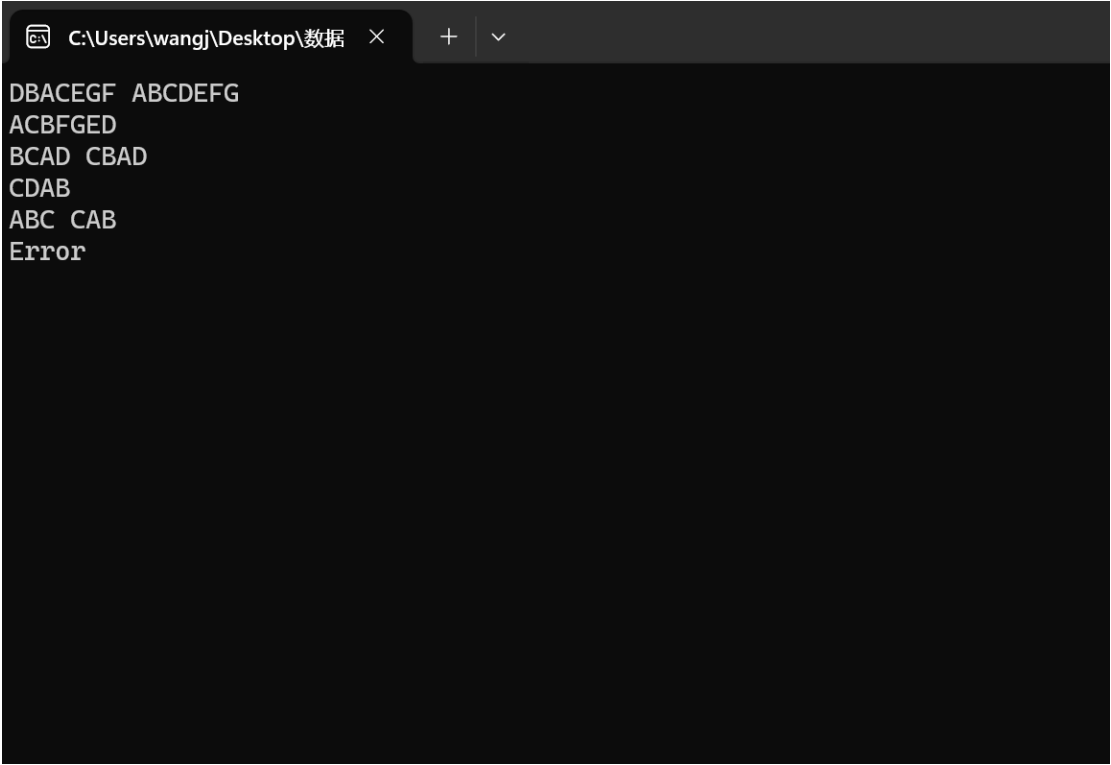
        string preorder = input.substr(0, spaceIndex);
        string inorder = input.substr(spaceIndex + 1);
        string postorder;

        if (BuildPostorder(preorder, inorder, postorder)) {
            cout << postorder << endl;
        }
        else {
            cout << "Error" << endl;
        }
    }

    return 0;
}

```

2.6.5 调试分析（遇到的问题解决方法）



```

C:\Users\wangj\Desktop\数据 >
DBACEGF ABCDEFG
ACBFGED
BCAD CBAD
CDAB
ABC CAB
Error

```

2.6.6 总结和体会

遍历遍历遍历!!! 递归递归递归!!! 通过递归, 可以简洁地处理树的分解和合并问题。

个 `children` 映射, 用于存储树的结构。`findLCA` 方法用于查找两个节点的最近公共祖先。

2.7 表达式树

2.7.1 问题描述

任何一个表达式, 都可以用一棵表达式树来表示。例如, 表达式 $a+b*c$, 可以表示为如下的表达式树:

```

+
 / \
a  *
   / \
  b  c
```

现在, 给你一个中缀表达式, 这个中缀表达式用变量来表示 (不含数字), 请你将这个中缀表达式用表达式二叉树的形式输出出来。

2.7.2 基本要求

输入格式:

输入分为三个部分。

第一部分为一行, 即中缀表达式(长度不大于 50)。

中缀表达式可能含有小写字母代表变量 ($a-z$), 也可能含有运算符 ($+$ 、 $-$ 、 $*$ 、 $/$ 、小括号), 不含有数字, 也不含有空格。

第二部分为一个整数 n ($n \leq 10$), 表示中缀表达式的变量数。

第三部分有 n 行, 每行格式为 $C \ x$, C 为变量的字符, x 为该变量的值。

对于 20% 的数据, $1 \leq n \leq 3$, $1 \leq x \leq 5$;

对于 40% 的数据, $1 \leq n \leq 5$, $1 \leq x \leq 10$;

对于 100% 的数据, $1 \leq n \leq 10$, $1 \leq x \leq 100$;

下载编译并运行 `p129_data.cpp` 生成随机测试数据

输出格式:

输出分为三个部分, 第一个部分为该表达式的逆波兰式, 即该表达式树的后根遍历结果。占一行。

第二部分为表达式树的显示, 如样例输出所示。

如果该二叉树是一棵满二叉树, 则最底部的叶子结点, 分别占据横坐标的第 1、3、5、7……个位置 (最左边的坐标是 1),

然后它们的父结点的横坐标, 在两个子结点的中间。

如果不是满二叉树, 则没有结点的地方, 用空格填充 (但请略去所有的行末空格)。

每一行父结点与子结点中隔开一行, 用斜杠 ($/$) 与反斜杠 ($\backslash$) 来表示树的关系。

$/$ 出现的横坐标位置为父结点的横坐标偏左一格, $\backslash$ 出现的横坐标位置为父结点的横坐标偏右一格。

也就是说, 如果树高为 m , 则输出就有 $2m-1$ 行。

第三部分为一个整数, 表示将值代入变量之后, 该中缀表达式的值。需要注意的一点是, 除法代

表整除运算，即舍弃小数点后的部分。
同时，测试数据保证不会出现除以 0 的现象。

2.7.3 数据结构设计

```
struct TreeNode {  
    char value;  
    TreeNode* left;  
    TreeNode* right;  
    TreeNode(char val) : value(val), left(nullptr), right(nullptr) {}  
};
```

2.7.4 功能说明（函数、类）

```
TreeNode* buildExpressionTree(const string& postfix) {  
    stack<TreeNode*> s;  
    for (char c : postfix) {  
        if (isalpha(c)) {  
            s.push(new TreeNode(c));  
        }  
        else {  
            TreeNode* node = new TreeNode(c);  
            node->right = s.top(); s.pop();  
            node->left = s.top(); s.pop();  
            s.push(node);  
        }  
    }  
    return s.top();  
}  
  
int getHeight(TreeNode* node) {  
    if (!node) return 0;  
    return 1 + max(getHeight(node->left), getHeight(node->right));  
}  
  
void fillTree(vector<vector<char>>& tree, TreeNode* node, int row, int col, int height) {  
    if (!node) return;  
    tree[row][col] = node->value;  
    if (node->left) {  
        tree[row + 1][col - (1 << (height - row / 2 - 2))] = '/';  
        fillTree(tree, node->left, row + 2, col - (1 << (height - row / 2 - 2)), height);  
    }  
    if (node->right) {  
        tree[row + 1][col + (1 << (height - row / 2 - 2))] = '\\';  
        fillTree(tree, node->right, row + 2, col + (1 << (height - row / 2 - 2)), height);  
    }  
}
```

```
}
```

```
void printTree(TreeNode* root) {  
    int height = getHeight(root);  
    int width = (1 << height) - 1;  
    vector<vector<char>> tree(2 * height - 1, vector<char>(width, ' '));  
    fillTree(tree, root, 0, width / 2, height);  
    for (const auto& row : tree) {  
        for (char c : row) {  
            cout << c;  
        }  
        cout << endl;  
    }  
}
```

```
int evaluateExpression(TreeNode* root, const unordered_map<char, int>& variables) {  
    if (!root) return 0;  
    if (isalpha(root->value)) {  
        return variables.at(root->value);  
    }  
    int left = evaluateExpression(root->left, variables);  
    int right = evaluateExpression(root->right, variables);  
    switch (root->value) {  
        case '+': return left + right;  
        case '-': return left - right;  
        case '*': return left * right;  
        case '/': return left / right;  
    }  
    return 0;  
}
```

```
int main() {  
    string infix;  
    cin >> infix;  
    int n;  
    cin >> n;  
    unordered_map<char, int> variables;  
    for (int i = 0; i < n; ++i) {  
        char var;  
        int value;  
        cin >> var >> value;  
        variables[var] = value;  
    }  
}
```

```

string postfix = infixToPostfix(infix);
cout << postfix << endl;

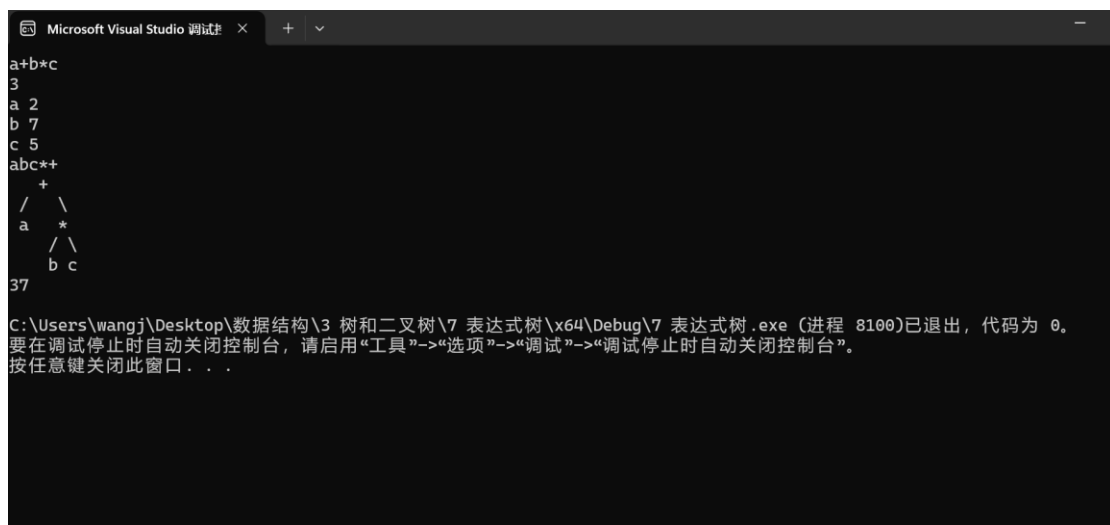
TreeNode* root = buildExpressionTree(postfix);
printTree(root);

int result = evaluateExpression(root, variables);
cout << result << endl;

return 0;
}

```

2.7.5 调试分析（遇到的问题解决方法）



2.7.6 总结和体会

- 中缀转后缀表达式：
 - 使用栈来处理操作符的优先级和括号。
 - 遇到操作数直接添加到后缀表达式中。
 - 遇到操作符时，根据优先级将栈顶操作符弹出并添加到后缀表达式中，直到栈为空或栈顶操作符优先级低于当前操作符。
 - 遇到左括号直接入栈，遇到右括号则弹出栈顶操作符直到遇到左括号。
- 构建表达式树：
 - 使用栈来存储节点。
 - 遇到操作数时，创建一个新节点并入栈。
 - 遇到操作符时，弹出栈顶的两个节点作为当前操作符节点的左右子节点，然后将当前操作符节点入栈。
- 表达式树的高度计算：
 - 递归计算左右子树的高度，取最大值加 1。
- 表达式树的打印：
 - 使用二维字符数组来表示树的结构。
 - 递归填充树的节点和连接符号（/和\）。

4. 实验总结

树是一种递归定义的数据结构。每棵树由一个根节点和若干子树组成，每个子树本身也是一棵树。由于树的这种递归性质，许多树的操作（如遍历、插入、删除等）都可以用递归来实现！